■ UFC FIGHT PREDICTION SYSTEM

Documentação Técnica de Engenharia e Modelagem Preditiva

Arquitetura de Software, Modelagem Estatística com XGBoost, Sistema Elo e Calibração de Probabilidades

Projeto: UFC Analysis & Fight Prediction	**Autor Original:** Arthur Santos
Dataset: 11.441 Lutas (1993–2026) | 4.581 Atletas	**Algoritmo:** XGBoost + Platt Scaling + Elo
Linguagem: Python 3.11+	**Status:** Produção / Calibrado e Testado

1. Visão Geral e Arquitetura do Sistema

Este projeto implementa uma solução ponta a ponta de aprendizado de máquina supervisionado para previsão probabilística de resultados de lutas no Ultimate Fighting Championship (UFC). A modelagem resolve três problemas clássicos em esportes de combate: o vazamento temporal de dados (data leakage), a assimetria e viés do corner vermelho (corner bias), e a hiperconfiança de probabilidade (overconfidence) inerente a árvores de decisão.

O pipeline processa 11.441 lutas históricas abrangendo o período de novembro de 1993 a agosto de 2026, transformando dados brutos em 47 atributos preditivos matematicamente formulados.

2. Stack Tecnológica e Bibliotecas Utilizadas

O projeto utiliza um ambiente virtual isolado (.venv) sob Python 3.11. Abaixo estão as bibliotecas essenciais e a responsabilidade de cada uma:

Biblioteca	Versão Mínima	Função no Sistema
xgboost	>= 2.0.0	Algoritmo central de Gradient Boosted Decision Trees (GBDT). Constrói 120 árvores de decisão aditivas focadas nos erros residuais.
scikit-learn	>= 1.3.0	Calibração de probabilidades via Platt Scaling (CalibratedClassifierCV), divisão temporal, cálculo de métricas (AUC-ROC, Brier Score, Acurácia).
pandas	>= 2.0.0	Manipulação vetorial de dados tabulares, junções cronológicas, parsing de datas e cálculos sequenciais.
numpy	>= 1.24.0	Operações matemáticas de alta performance, cálculo de probabilidades logísticas e métricas numéricas.
kagglehub	>= 0.2.0	Download programático e sincronização automática das versões do dataset hospedado no Kaggle.
pytest	>= 7.0.0	Suíte de testes automatizados de regressão, integridade de arquivos e verificação de simetria matemática.
reportlab	>= 4.0.0	Geração programática e estilização deste relatório técnico oficial em PDF.

3. Modelagem Estatística e Algoritmos

3.1. O Sistema de Rating Elo Sequencial

Diferente de modelos convencionais que usam apenas médias estáticas de carreira, o sistema calcula o rating dinâmico de cada atleta estritamente antes do gongo inicial de cada luta. A habilidade é modelada como uma variável aleatória latente cuja probabilidade de vitória esperada E_A entre dois lutadores com ratings R_A e R_B segue a curva logística:

```
Equação de Probabilidade Esperada:   E_A = 1 / (1 + 10^((R_B - R_A) / 400))
Regra de Atualização Sequencial:   R_A' = R_A + K * (S_A - E_A)
```

Onde K = 32 e S_A representa o resultado empírico (1 para vitória, 0 para derrota, 0.5 para empate). Os ratings iniciam em 1.500 para estreantes.

3.2. Extreme Gradient Boosting (XGBoost)

O classificador minimiza a função de perda de entropia cruzada binária (Log-Loss) através de aproximações polinomiais de Taylor de segunda ordem. Configuração dos hiperparâmetros de regularização:

- **n_estimators:** 120 árvores | **max_depth:** 4 (árvores rasas para evitar sobreajuste)
- **learning_rate:** 0.05 | **subsample:** 0.8 | **colsample_bytree:** 0.8
- **min_child_weight:** 3 | **gamma:** 0.5 (penalidades severas contra nós espúrios)

3.3. Calibração Probabilística de Platt (Platt Scaling)

Modelos de árvores sofrem de hiperconfiança: os scores brutos tendem a se agrupar perto de 0% e 100%. Para solucionar isso, encapsulamos o XGBoost em um calibrador sigmoidal (CalibratedClassifierCV com 5-fold cross validation), ajustando a regressão logística P(y=1 | f(x)) = 1 / (1 + exp(A*f(x) + B)). Isso eliminou previsões irreais acima de 90%, mantendo a média em ~64-67%.

4. Engenharia de Atributos (Vetor de 47 Features)

O modelo avalia 47 variáveis numéricas padronizadas para cada confronto, agrupadas em seis pilares técnicos:

Grupo	Qtd	Variáveis / Atributos	Importância Técnica
Rating Elo	4	r_elo_before, b_elo_before, elo_diff, elo_expected_r	Nível competitivo e momento histórico estritamente anterior à luta.
Físico	6	r/b_height, r/b_weight, r/b_reach, height_diff, reach_diff, weight_diff	Vantagens de alavancagem, envergadura e categoria de peso oficial.
Striking	8	r/b_splm, r/b_str_acc, r/b_sapm, r/b_str_def, splm_diff, str_acc_diff, sapm_diff, str_def_diff	Volume de golpes por minuto, acurácia e capacidade defensiva de esquiva.
Grappling	8	r/b_td_avg, r/b_td_avg_acc, r/b_td_def, r/b_sub_avg, td_avg_diff, td_def_diff	Ameaça de quedas, taxa de finalização e defesa de quedas (TDD).
Carreira	8	r/b_wins, r/b_losses, win_diff, loss_diff, r/b_win_rate, win_rate_diff	Bagagem no octógono e aproveitamento percentual pré-luta.
Momento	3	r_streak, b_streak, streak_diff	Sequência ativa de vitórias ou derrotas consecutivas ao entrar no cage.
Diferenciais	10	Subtrações relativas normalizadas entre lutador A e lutador B	Permite simetria matemática e isolamento de vantagens relativas.

4.1. Decisão Metodológica: O Paradoxo da Idade em Campeões de Elite

A idade bruta (age_diff) foi propositalmente omitida do modelo final. Testes empíricos demonstraram que, embora na média da população de 4.581 atletas os lutadores mais velhos sofram declínio, a inclusão linear da idade causava 'etarismo artificial' (age bias): transformava lendas comprovadamente dominantes como Alex Pereira (39 anos) e Alexander Volkanovski (38 anos) em zebras injustificadas contra adversários mais jovens. O sistema optou por capturar o declínio físico de forma justa através da queda do Elo e da sequência de vitórias (streak), protegendo os outliers de elite.

5. Metodologia de Validação e Simetria de Corners

• **Divisão Temporal Estrita (Out-of-Time):** O dataset foi ordenado por data e dividido temporalmente. O conjunto de treinamento utiliza 6.636 lutas realizadas até 31/12/2022. O conjunto de teste utiliza 2.143 lutas realizadas de 01/01/2023 a agosto de 2026. Isso garante que o modelo nunca treine com informações do futuro.

• **Data Augmentation Simétrico:** Historicamente no UFC, o Red Corner é reservado ao favorito ou campeão, vencendo >60% das lutas apenas pelo pareamento da organização. Para eliminar esse viés, cada amostra de treino é duplicada e invertida (Red vira Blue, diferenciais invertem o sinal e y vira 1-y), totalizando 13.272 amostras de treino. Isso garante a propriedade de invariância: P(A vence B) + P(B vence A) = 1.0.

6. Comparativo Estatístico e Resultados de Performance

Métricas avaliadas no conjunto cego de teste temporal (2.143 combates reais entre 2023 e 2026):

Métrica Estatística	Repositório Original	Modelo Final Calibrado	Impacto da Melhoria
Acurácia Out-of-Time	60.0% – 65.0%	**70.2%**	Superioridade de +5% a 10% em dados estritamente futuros.

Área sob a Curva (AUC-ROC)	0.60 – 0.65	**0.7744**	Forte separabilidade probabilística entre vencedores e perdedores.
Brier Score (Erro Quadrático)	~0.2450 (alto erro)	**0.1952 / 0.2066**	Redução significativa no desvio probabilístico médio.
Confiança Média	83.0% (irreal)	**60.0% – 64.7%**	Totalmente alinhado à taxa real de vitória de favoritos no MMA (~65%).
Predições com Certeza > 90%	37.7% das lutas	**0.0%**	Eliminação total do problema de hiperconfiança infundada.
Lutas Pareadas (50% a 60%)	7.3% das lutas	**54.5%**	Reconhecimento fiel do equilíbrio e da volatilidade do esporte.

7. Estrutura Modular dos Scripts e Responsabilidades

Arquivo	Tipo	Responsabilidade no Sistema
update_dataset_2026.py	Pipeline ETL	Ingere master.csv e fighter.csv do Kaggle, converte pés/polegadas para cm e libras para kg, e calcula histórico dinâmico pré-luta.
calculate_elo.py	Motor Estatístico	Executa a formulação de ratings Elo cronológicos para 4.581 atletas ao longo de 11.441 combates sem lookahead bias.
feature_engineering.py	Extrator de Dados	Padroniza o vetor de 47 atributos e gera as matrizes de treinamento simétricas (Corner Invariance).
train_xgboost.py	Treinamento ML	Treina o modelo XGBoost, calibra as probabilidades via Platt Scaling e exporta o binário 'ufc_xgboost_model.pkl'.
test_predictions.py	CLI de Produção	Interface de linha de comando para predição em tempo real com validação de categorias de peso oficiais do UFC.
run_tests.py	QA / Testes	Executa bateria completa de 6 testes automatizados (arquivos, dados, inferência, simetria e regras de categoria).
data_loader.py	Utilitário	Abstrai caminhos de arquivo relativos e garante o download automático das bases de dados se ausentes.